

Contents

What is .fsf in Software Heritage? A non-programming extension study	1
Abstract	1
1. What .fsf is (evidence)	1
2. Method (reused toolkit)	1
3. Results	2
4. Insights	6
5. Limitations	6
6. Reproducibility & artefacts	6

What is .fsf in Software Heritage? A non-programming extension study

Second application of the extension-study playbook (docs/swh_extension_study_playbook.md), on a non-PL extension. Design: docs/fsf_study_design.md. Toolkit: tools/fsf/. Companion: the COBOL study docs/cobol_swh_study.md.

Abstract

We characterise what Software Heritage indexes under the .fsf extension, using the same pipeline as the COBOL study (sample -> fetch -> indicators -> enum LLM-judge -> reclassifier -> origins) but asking “*what is this content, and which languages/notations does it relate to?*” rather than “is it language X”. Across **1,662 LLM-judged contents** (a uniform-random and an origin-diverse sample from 21,800 contents / 757 repos) we find .fsf is **not a programming language (99 %)**: it is overwhelmingly the **FSL FEAT design file** — an fMRI neuroimaging analysis *configuration*, expressed in **Tcl set fmri(...)** syntax (Tcl 77 %) — from the neuroimaging community (OpenNeuro, DiedrichsenLab, FMRIB). A deterministic **set fmri(** marker matches the judge on “is FEAT” for **100 %** of judged files. The reframed question pays off: the extension is **polysemous** — its tail relates to genuinely different languages, most strikingly **GLSL fragment shaders** (.fsf = *fragment shader file*) and C++/GLSL headers, plus a large **git-annex pointer** artefact from DataLad data-management. The method is thus demonstrated to work symmetrically on a real PL (COBOL) and a domain config (.fsf).

1. What .fsf is (evidence)

A fetched sample:

```
# FEAT version number
set fmri(version) 3.14
set fmri(inmelodic) 1
set fmri(level) 1
set fmri(analysis) 7
set fmri(outputdir) "..."
```

FSL FEAT (fMRI Expert Analysis Tool, part of the FMRIB Software Library) design files: heavily-commented Tcl set fmri(...) configs, usually GUI-generated. Filenames confirm it (design.fsf, feat_level_func, *.fmri.task*.run.*, BIDS sub-*_run-*). It is a **domain-specific configuration**, not a general-purpose language — so a good *control* case for the method.

2. Method (reused toolkit)

Identical pipeline to COBOL, reparameterised in tools/fsf/ (taxonomy, indicators — set fmri(counts, level, inmelodic, EVs, comment ratio; reclassify — the set fmri(marker; judge — the fsf-judge/1 enum schema; study — uniform + origin-diverse sampling; make_figures; review_app). SWH fetch/cache and the OpenRouter judge plumbing are reused from tools/cobol. Two frames: **uniform-random 1,000** (file population) and **origin-diverse 757** (<=1 file per repo — the project population). 1,662 judged (982 uniform + 706 diverse), **\$30.9**.

The judge is asked to lead with **content_type** + free-text format, **expressed_in**, **related_languages[]**, **ecosystem_tool** — the substance — with **is_programming_language** and FEAT-specifics as secondary detail.

3. Results

3.1 Is .fsf a programming language? — No (99 %)

`is_programming_language = False` for 1,641 / 1,662 (99 %). The method correctly declines to call an FSL FEAT config (or its Tcl host syntax) “a programming language”. The 21 `True` cases are exactly the non-FEAT tail — GLSL shaders and C++/GLSL headers (§3.4).

3.2 What it is, and what it relates to

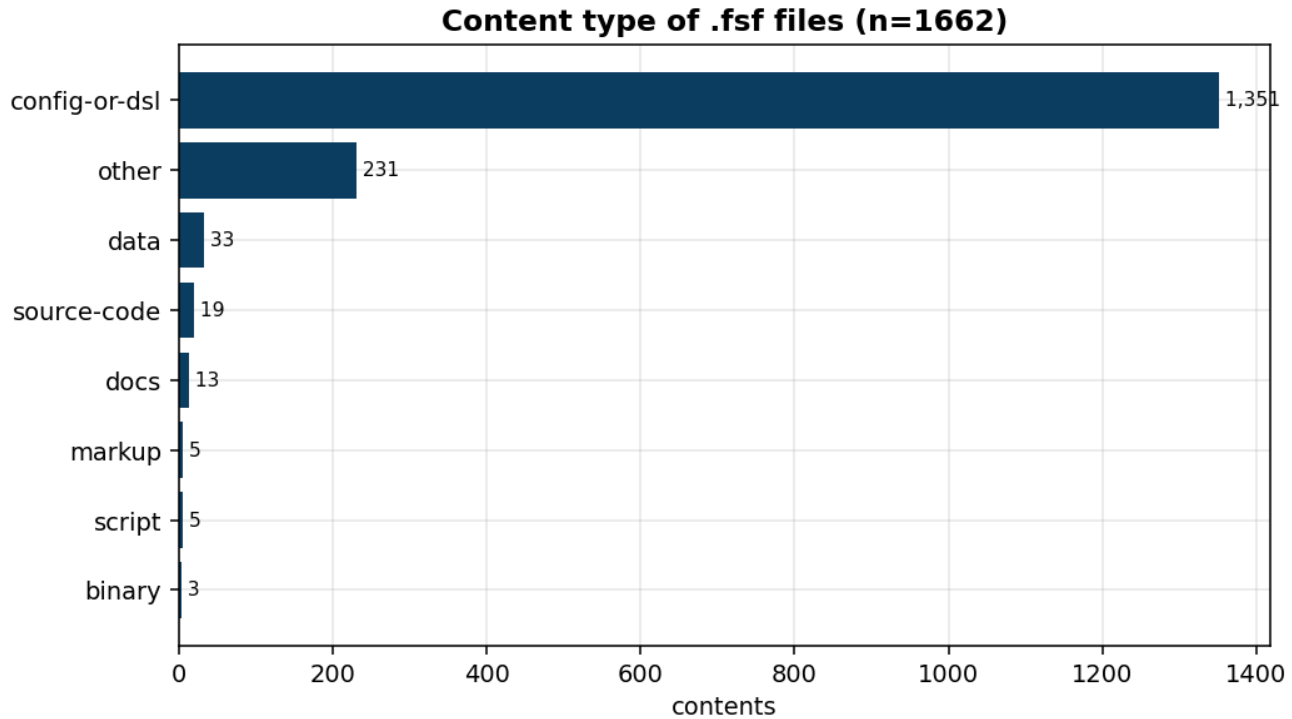


Figure 1: Content type

Content type: `config-or-dsl` 76–85 %, `other` 13–16 %, small `data/docs/source-code/markup` tails. **Format:** “FSL FEAT design file” dominates (678 uniform + 403 diverse), with FEAT/MELODIC and template variants. **Ecosystem:** FSL / FEAT (FMRIB).

Related languages (the polysemy axis): **Tcl** 77 % (the FEAT host syntax), then a real tail — **XML** 57, **GLSL** 11, **JSON** 7, **C++** 6, **ML** 6, **HTML/CSS/JS** 5 each, **C** 5, **OCaml/Haskell** 4. So `.fsf` *relates to* Tcl by construction, but across the corpus it also relates to shader, markup, and functional languages.

3.3 FSL FEAT sub-types

Of the FEAT files: **first-level** analyses dominate (55–69 %) over higher-level (12–16 %); **task-GLM** (48–62 %) over resting-state/MELODIC-ICA (7 %) and group-stats (12–16 %); **GUI-generated** (54–76 %) over hand-edited. FEAT version is almost entirely the 3.x/6.x line.

3.4 The polysemy / contamination tail

The `other` slice (13–16 %) and the non-Tcl `related_languages` are where `.fsf` is *not* FSL FEAT:

- **GLSL fragment shaders** — `.fsf` is also a *fragment-shader* extension. `2DShader.fsf`, `ppFragment.fsf` are GLSL; `compose.fsf`, `dither.fsf`, `hband.fsf` are C++ headers embedding GLSL. (These are ~all the `is_programming_language = True` cases.)

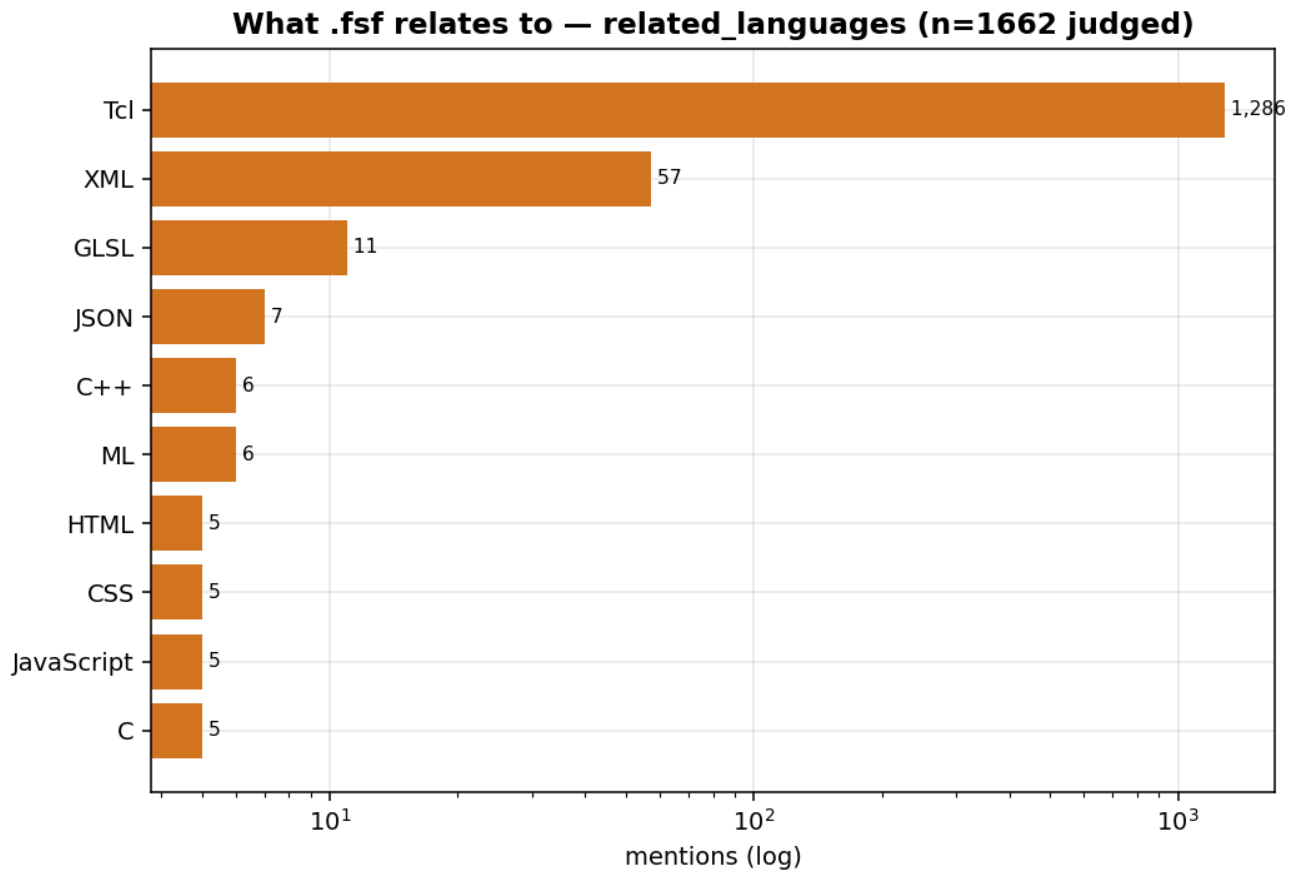


Figure 2: What .fsf relates to

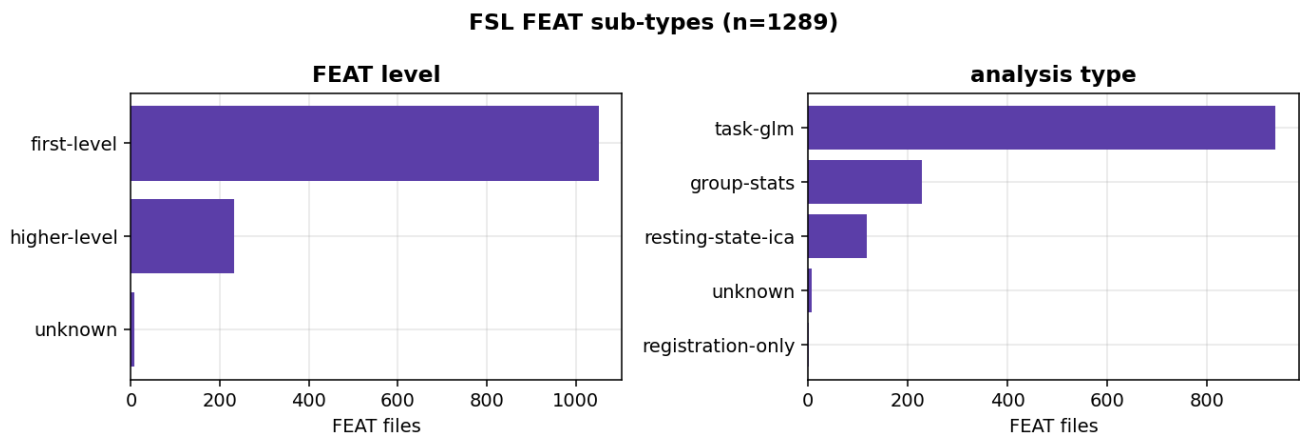


Figure 3: FEAT sub-types

- **git-annex / DataLad pointers** — a *large* artefact: ~120 files per frame are git-annex symlink/pointer stubs, not FEAT content. In DataLad-managed neuroimaging datasets the real `.fsf` is annexed, so what SWH archived under that path is a pointer. This is a data-management contamination specific to the neuroimaging ecosystem.
- Odd tails: an “unknown fractal rendering application” (17), misc XML/HTML/JSON.

3.5 By-file vs by-repo

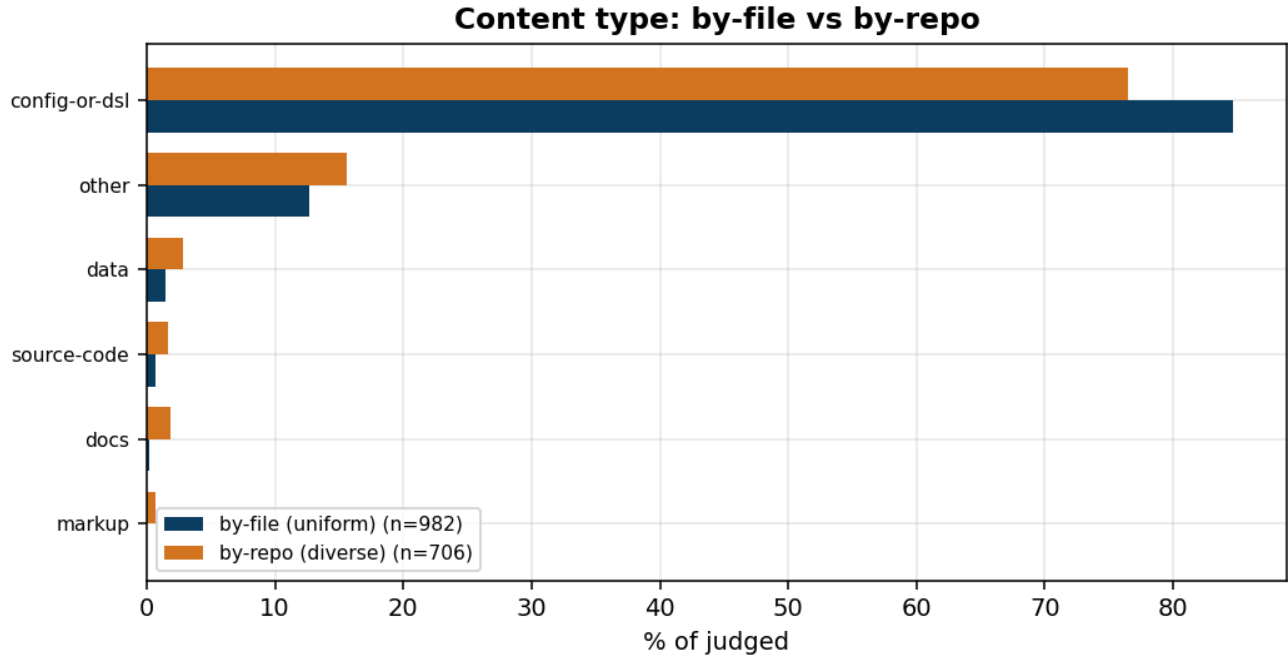


Figure 4: Content type: by-file vs by-repo

	uniform (by-file, n=982)	diverse (by-repo, n=706)
has FEAT marker	80 %	67 %
config-or-dsl	85 %	76 %
other (non-FEAT)	13 %	16 %
GUI-generated	76 %	54 %
hand-edited	6 %	22 %
distinct origins	173	757
median lines	419	392

Same lesson as COBOL, milder: sampling **by repo** surfaces *more* diversity — more non-FEAT **other**, and far more **hand-edited** designs (22 % vs 6 %) — while by-file over-weights the auto-generated designs that big studies emit in bulk (999 files from just 173 repos).

3.6 Provenance

21,800 contents from 757 repos, led by GitHub with a strong neuroscience signal (`git.mpib-berlin.mpg.de` = Max Planck). Unlike COBOL, **no synthetic fixture dominates**: the top repo is **18 %** (a real study, `QQXiao/ISR_2015`) vs COBOL’s WBC fixture at 40 %. The top repos are pure neuroimaging — **OpenNeuroDatasets** (the fMRI/BIDS dataset hub), `DiedrichsenLab`, `Opioid_Reward_fMRI`, fMRI-task repos (`flashtask`, `PacManStim`). `.fsf` on SWH is a **real research corpus**, its concentration driven by studies auto-generating one design per subject x run.

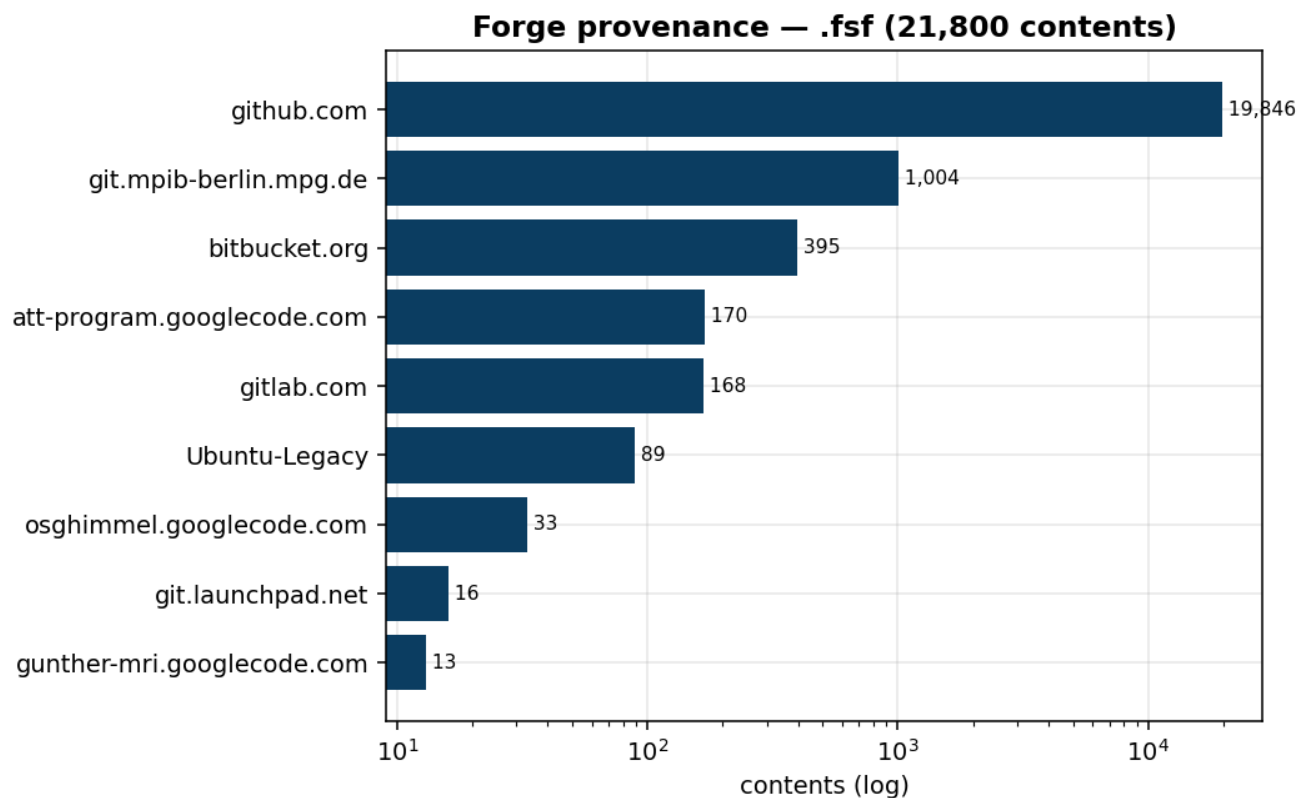


Figure 5: Forge provenance

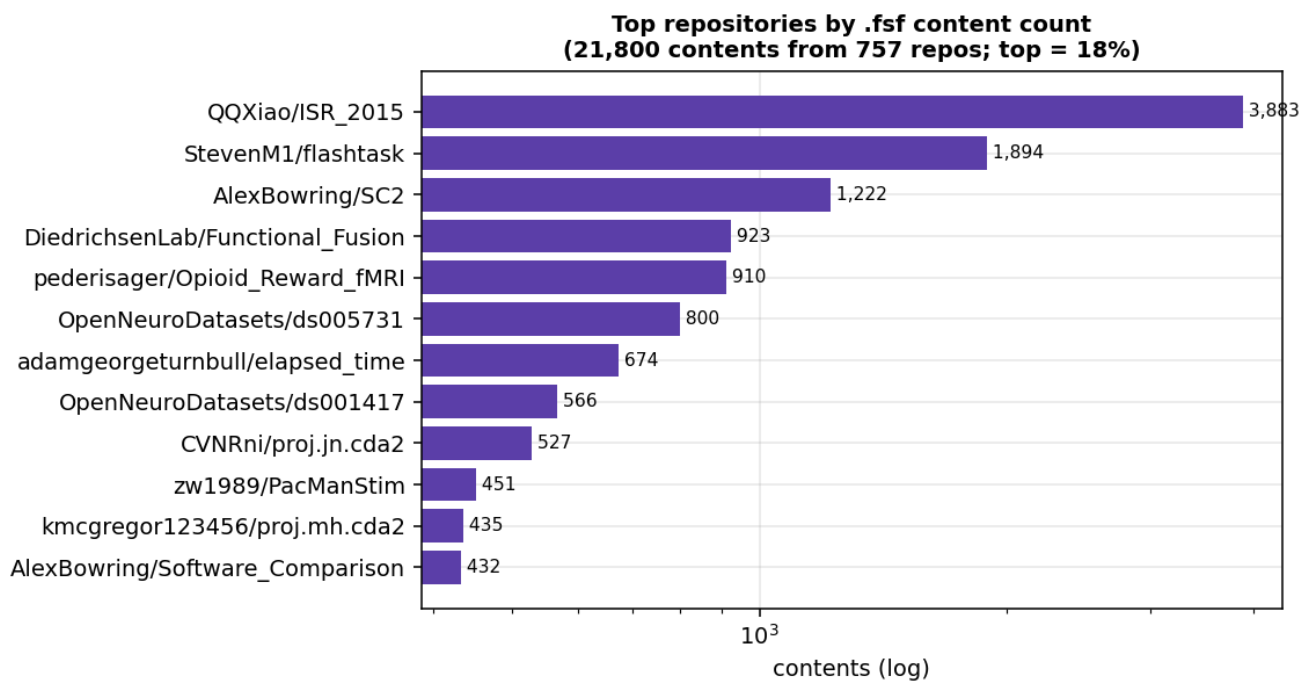


Figure 6: Top repositories

3.7 Reclassifier validation

The zero-API reclassifier (label = has `set_fmri()`) agrees with the LLM judge on “is this FEAT” for **1,654 / 1,662 = 100 %** — a much cleaner marker than COBOL’s divisions (F1 0.88), as predicted for a single-token signature. So the FEAT vs non-FEAT split can be computed archive-wide at zero cost.

4. Insights

1. **The method generalises to a non-PL.** It correctly says `.fsf` is *not* a programming language (99 %) and names it (FSL FEAT config, Tcl, neuroimaging) — the negative-case validation the design called for.
2. **“What relates to what” beats “is it language X”.** The single most informative output is the `related_languages` histogram: Tcl by construction, but GLSL/C++/ML in the tail. A binary is-PL question would have hidden the `.fsf` = *fragment shader* polysemy entirely.
3. **Contamination is ecosystem-specific.** COBOL’s noise was a synthetic fixture + comic-book lists; `.fsf`’s is **git-annex pointers** (DataLad) and **GLSL shaders**. Each extension carries its own kind of non-target content; only content inspection finds it.
4. **A cleaner target than COBOL.** One-token `set_fmri()` gives 100 % reclassifier/judge agreement; the by-file/by-repo gap is milder (no fixture). `.fsf` is a coherent, mostly-clean domain corpus.

5. Limitations

- **git-annex pointers** inflate the FEAT-looking share at the *file* level and under-represent actual design content (the real files are annexed off-SWH).
- The 21 non-FEAT PLs are a small sample of the shader/other tail; a targeted sweep of non-`set_fmri()` `.fsf` would size it better.
- Judge accuracy is validated against the deterministic marker (100 % on is-FEAT) but, as in COBOL, human ground truth is future work — the fsf review app (`tools/fsf/review_app.py`, with group-assertion rules) exists for it.
- 757 origins is the whole by-repo ceiling for this extension; larger scale needs no new origins (the pool is small).

6. Reproducibility & artefacts

```
python3 -m tools.fsf.study --sample --n 1000 --seed 5
SWH_TOKEN=... OPENROUTER_API_KEY=... python3 -m tools.fsf.study --run --judge
python3 -m tools.fsf.study --report
python3 -m tools.fsf.make_figures
python3 -m tools.fsf.review_app                # http://127.0.0.1:8767
```

Artefacts: `data/derived/fsf_study/` (`worklist_all.csv`, `reports/<sha>.json`, `summary_fsf.json`), `fsf_files+origin.csv` (graph provenance), figures under `docs/assets/fsf/`. Toolkit: `tools/fsf/`.

Judge: *claude-sonnet-4.6* · 1,662 judged · \$30.9 · study authored with Claude Code.